AWS SAA-C03 Study Guide

Module 2
Resilient Architectures

Scaling & statelessness • Load balancing • Auto Scaling mechanics • Decoupling with queues & events • Route 53 • Database HA • Disaster recovery • Worked practice questions

1. Resilience Concepts: The Vocabulary That Decides Answers

1.1 High availability vs fault tolerance

High availability (HA) means the system keeps working with minimal downtime when a component fails — there may be a brief interruption while failover happens (RDS Multi-AZ takes ~60–120 seconds). **Fault tolerance** is stricter: users notice NOTHING because redundant capacity is already serving traffic (an ASG across 3 AZs behind an ALB — lose an AZ, the other two just keep answering). Fault tolerance costs more because you pay for the redundancy continuously. When a question distinguishes the two, it's testing whether you'll pay for standby-that-serves vs standby-that-waits.

1.2 Vertical vs horizontal scaling, and why statelessness matters

Vertical scaling = a bigger instance. Simple, but has a ceiling, requires downtime to resize, and the single machine is still a single point of failure. **Horizontal scaling** = more instances. Near-limitless and inherently HA — but only if any instance can serve any request. That requires the app to be **stateless**: no user session data, uploaded files, or local caches living on one box.

CORE CONCEPT — Externalizing state: The recurring fix pattern: sessions → ElastiCache or DynamoDB; uploaded files → S3 (or EFS if the app truly needs a shared POSIX filesystem); logs → CloudWatch/Kinesis. Any question where 'users lose their session when an instance is replaced' or 'uploads are missing on some servers' is asking you to move that state off the instance.

1.3 RPO and RTO

RPO (Recovery Point Objective): how much DATA you can afford to lose, measured backward in time from the disaster ('RPO 1 hour' → backups/replication at least hourly). **RTO (Recovery Time Objective):** how long you can afford to be DOWN. These two numbers, plus budget, select the DR strategy (Section 8). Train yourself to extract them from every scenario — they're usually stated explicitly.

1.4 Loose coupling

Tightly coupled systems call each other directly and synchronously — if the downstream is slow or dead, the upstream fails too, and a traffic spike crushes everything at once. **Loose coupling** inserts a buffer (queue, topic, event bus) between components so each side scales and fails independently. This one idea generates most of Domain 2's questions: web tier drops work into SQS; workers consume at their own pace; a worker crash loses nothing because the message becomes visible again.

2. Elastic Load Balancing In Depth

2.1 What a load balancer actually does

An ELB is a managed fleet of traffic distributors deployed across the AZs you choose. Clients connect to the ELB's DNS name; it forwards to healthy registered targets in **target groups** (instances, IPs, Lambda, containers). Health checks probe each target; failures pull a target out of rotation. This gives you the fault-tolerant front door: an AZ dies, the ELB simply stops sending traffic there.

2.2 The three load balancers that matter

	Application LB (ALB)	Network LB (NLB)	Gateway LB (GWLB)
OSI layer	7 — understands HTTP/HTTPS/gRPC	4 — TCP/UDP/TLS, doesn't read HTTP	3 — GENEVE encapsulation

	Application LB (ALB)	Network LB (NLB)	Gateway LB (GWLB)
Superpower	Content-based routing: by path (/api/*), host (shop.example.com), headers, query strings; redirects; fixed responses; authenticates users via Cognito/OIDC before traffic hits the app	Extreme performance (millions rps), ultra-low latency, ONE STATIC IP (or your Elastic IP) PER AZ, preserves client source IP, handles any TCP/UDP protocol	Transparently inserts 3rd-party appliances (firewalls, IDS/IPS, DPI) into the traffic path
Typical targets	ECS services (dynamic ports), EC2, Lambda	EC2, IPs, an ALB (static IPs in front of L7 routing), PrivateLink endpoint services	Appliance fleets
Choose when	Web/microservice HTTP routing	Static IPs for firewall whitelisting; non-HTTP; extreme throughput; source-IP preservation	'Inspect all traffic with a third-party appliance'

- Client IP: ALB terminates the connection, so apps read the original client from the `X-Forwarded-For` header. NLB (instance targets) passes the source IP through natively.
- **Cross-zone load balancing:** distributes evenly across ALL targets in all AZs rather than splitting per-AZ first. Always on and free for ALB; off by default (and charged) for NLB. Uneven load with unequal targets per AZ → enable cross-zone.
- **Sticky sessions** pin a client to one target via cookie — a crutch for stateful apps. It skews load and breaks when the target dies; the better answer in design questions is externalized session state.
- **Connection draining / deregistration delay:** lets in-flight requests finish before a target is removed — pairs with Auto Scaling scale-in.

EXAM TIP: 'Company firewall must whitelist fixed IPs' → NLB (or Global Accelerator in front of an ALB). 'Route /video to one fleet and /api to another' → ALB rules. 'Millions of UDP packets per second' → NLB.

3. Auto Scaling Mechanics

3.1 The moving parts

An **Auto Scaling group** maintains between **min** and **max** instances, targeting **desired** capacity, spread across the subnets/AZs you list. A **launch template** tells it what to launch (AMI, instance type(s), SG, role, user data — templates are versioned and support mixed instance types + Spot; legacy 'launch configurations' are immutable and deprecated). The ASG continuously: (1) replaces instances that fail health checks — self-healing; (2) rebalances across AZs; (3) executes scaling policies.

- Health checks: EC2 status checks by default; turn on **ELB health checks** so an instance whose APP is dead (but VM is fine) also gets replaced — a favorite exam nuance.

3.2 Scaling policies — choose by scenario wording

Policy	How it works	Trigger phrase
Target tracking	You state a target (avg CPU 50%, requests/target 1000); AWS does the alarm math both directions	Default best answer: 'keep utilization around X'
Step scaling	CloudWatch alarm thresholds add/remove capacity in steps you define	'Fine-grained control over scaling increments'
Scheduled	Scale at set times	'Traffic spikes every weekday at 9 AM'
Predictive	ML forecasts recurring load and scales AHEAD of it	'Cyclical/recurring patterns; capacity ready BEFORE the spike'

- **Cooldown/warm-up** stop the ASG reacting to its own in-progress changes (thrashing). **Lifecycle hooks** pause launch/terminate to run setup or log-draining. **Instance warm pools** keep pre-initialized stopped instances for apps with slow boot.

- **Worker-fleet pattern:** scale on a CUSTOM metric — SQS queue depth ÷ instance count ('backlog per instance') — because CPU says nothing about how much work is queued. Appears constantly.

EXAM TIP: Scale-in protection + lifecycle hooks answer 'instances processing long jobs must not be terminated mid-job.' Golden AMI (pre-baked image) answers 'instances take too long to become healthy after scale-out' — bake dependencies in instead of installing via user data.

4. Decoupling: Queues, Topics, Events, Workflows

4.1 SQS — the buffer

A producer writes a message; it's stored redundantly across AZs; a consumer POLLS, receives it, and the message becomes invisible for the **visibility timeout** (default 30s). If the consumer finishes, it DELETES the message; if it crashes, the timeout expires and the message reappears for another consumer — that's the durability model, and it implies **at-least-once delivery**: design consumers to be idempotent (safe to process twice).

- **Standard queues:** unlimited throughput, best-effort ordering, occasional duplicates. **FIFO queues** (name ends .fifo): strict order within a MessageGroupId and exactly-once processing, at 300 msg/s (3,000 batched; more in high-throughput mode).
- **Long polling** (WaitTimeSeconds up to 20): the receive call waits for messages instead of returning empty — fewer API calls, lower cost, lower latency. Almost always 'enable long polling' when SQS cost/empty-receives appear.
- **Dead-letter queue (DLQ):** after maxReceiveCount failed attempts, the message moves to a side queue for inspection — poison messages stop clogging the pipeline. Retention up to 14 days (default 4). Max size 256 KB — for bigger payloads store in S3 and enqueue the pointer.

4.2 SNS — the megaphone

Pub/sub PUSH: publishers send to a **topic**; SNS immediately pushes copies to every subscriber (SQS, Lambda, HTTP, email, SMS, mobile push). SNS stores nothing after delivery — so the canonical **fan-out pattern** pairs it with SQS: one 'OrderPlaced' event → topic → three queues (billing, shipping, analytics), each processed independently, durably, at its own pace. **Filter policies** let each subscription receive only matching messages (e.g., only orders > $100).

4.3 EventBridge — the router

A serverless event bus. Sources: nearly every AWS service, your apps, and SaaS partners. **Rules** pattern-match events' JSON and route to 20+ target types (Lambda, Step Functions, SQS, ECS tasks, another bus in another account...). Also does cron **schedules**, event **archive & replay**, and a schema registry. Default answer for 'when X happens in service A, trigger B' and cross-account event routing. SNS = high-fanout push to subscribers; EventBridge = content-based routing between services.

4.4 Step Functions & the rest

- **Step Functions:** serverless state machines orchestrating multi-step workflows — sequences, parallel branches, choices, and crucially built-in RETRY and error handling per step. 'Coordinate multiple Lambda functions with retries/human approval' → Step Functions, not Lambda-calling-Lambda. Standard workflows run up to 1 year; Express for high-volume short bursts.
- **Amazon MQ:** managed ActiveMQ/RabbitMQ. ONLY correct when migrating an app already speaking AMQP/MQTT/JMS/STOMP that can't be rewritten. New cloud-native designs → SQS/SNS.

EXAM TIP: Buffer & smooth spikes → SQS. One event, many independent consumers → SNS→SQS fan-out. React to AWS service events / route by content / schedule → EventBridge. Multi-step logic with retries → Step Functions. Legacy broker protocols → Amazon MQ.

5. Route 53: DNS as a Resilience Tool

5.1 DNS in one paragraph

DNS translates names to IPs. Route 53 hosts your domain's records in a **hosted zone** — public (answers the internet) or private (answers only inside associated VPCs). Records: A (name→IPv4), AAAA (IPv6), CNAME (name→another name; NOT allowed at the zone apex like example.com). **Alias records** are Route 53's special type: they point at AWS resources (ALB, CloudFront, S3 website, API Gateway), work at the apex, are free to query, and track the target's IP changes automatically — prefer Alias over CNAME for AWS targets every time.

5.2 Routing policies — the resilience levers

Policy	Behavior	Scenario trigger
Simple	One record → value(s); no health checks	Single endpoint
Weighted	Split by assigned weights	Blue/green, canary (send 5% to new version), gradual migration
Latency	Answer with the Region fastest for THIS user	Multi-Region active-active performance
Failover	Primary while healthy; secondary when its health check fails	Active-passive DR (secondary is often an S3/CloudFront static 'sorry' page)
Geolocation	Route by user's country/continent; set a default	Compliance ('EU users stay in EU'), localized content
Geoproximity	Geographic routing with a tunable BIAS to grow/shrink a Region's share	'Shift more traffic toward Region X gradually'
Multivalue answer	Up to 8 healthy records returned; client picks	Cheap DNS-level load spreading with health checks

- **Health checks** can monitor an endpoint from multiple global checkers, monitor OTHER health checks (calculated — 'alarm if 2 of 3 fail'), or monitor a CloudWatch alarm (the only way to health-check private resources). Attach them to failover/weighted/latency records to stop routing to dead Regions.
- DNS caveat: clients cache answers for the TTL, so DNS failover isn't instant — low TTLs speed it up at the cost of more queries. (Global Accelerator, Module 4, fails over without DNS changes.)

6. Database High Availability & Read Scaling

6.1 RDS: Multi-AZ vs Read Replicas — the classic confusion

	Multi-AZ (availability)	Read Replica (performance)
Replication	SYNCHRONOUS — commit lands on standby before success returns; zero data loss	ASYNCHRONOUS — replica lags seconds behind (eventual consistency)
Readable?	Standby NOT readable (classic Multi-AZ instance). Newer 'Multi-AZ DB cluster' = 1 writer + 2 READABLE standbys	Yes — that's the point; up to 15; app must send reads to replica endpoints
Failover	Automatic, ~60–120 s; SAME DNS endpoint — no app change	None automatic; you can PROMOTE a replica manually (a DR maneuver)
Placement	Second AZ, same Region	Same AZ, cross-AZ, or CROSS-REGION
Solves	'Survive AZ failure with no data loss'	'Read-heavy workload / reporting queries slow down production'

- They combine: production = Multi-AZ AND read replicas (replicas can themselves be Multi-AZ). Reporting/BI workloads → point them at a replica so production writes are untouched.

6.2 Aurora — what makes it different

Aurora (MySQL/PostgreSQL-compatible) separates compute from a shared storage layer that keeps **6 copies of every block across 3 AZs** (writes need 4/6 — survives an entire AZ + one more copy failing) and auto-grows to 128 TB. Because replicas read the SAME shared storage, up to **15 replicas** run with ~10ms lag, and failover promotes one in <30 s. Apps use the **writer endpoint** and the load-balanced **reader endpoint**.

- **Aurora Global Database:** storage-level replication to up to 5 secondary Regions at ~1 s lag → RPO ~1 s, RTO <1 min. THE answer for aggressive cross-Region relational DR/reads.
- **Aurora Serverless v2:** capacity scales in fine increments with load — variable, spiky, or dev/test workloads without capacity planning.

6.3 DynamoDB and ElastiCache resilience

- **DynamoDB** replicates across 3 AZs automatically — Multi-AZ is free and built in. **Global Tables** = multi-Region ACTIVE-ACTIVE (read AND write in every Region, last-writer-wins conflicts). Point-in-time recovery restores to any second in 35 days.
- **ElastiCache Redis:** primary + replicas, Multi-AZ automatic failover, snapshots. Memcached: no replication, no persistence — a cache you can afford to lose. 'Cache must survive node failure' → Redis.
- **RDS Proxy:** managed connection pool in front of RDS/Aurora — absorbs Lambda connection storms and cuts failover time (~66%) by holding connections through it. 'Lambda exhausts DB connections' → RDS Proxy.

7. Backups & Snapshots

- **EBS snapshots:** incremental, stored in S3, copyable cross-Region/account — the building block of instance-level DR. **AMIs** = bootable snapshots; golden AMIs speed recovery and scale-out.
- **RDS/Aurora:** automated backups (transaction logs every 5 min → point-in-time restore within retention, max 35 days) vs manual snapshots (kept until you delete; survive instance deletion; shareable cross-account). Restores always create a NEW instance/endpoint.
- **AWS Backup:** one policy-driven service covering EBS, RDS/Aurora, DynamoDB, EFS, FSx, S3, Storage Gateway...: schedules, retention, lifecycle to cold storage, CROSS-REGION and CROSS-ACCOUNT copies, and **Vault Lock** — WORM-immutable backups nobody (including root) can delete. 'Centralize backups across services/accounts' or 'ransomware-proof backups' → AWS Backup (+ Vault Lock).

8. Disaster Recovery Strategies

Region-level DR is a spectrum: pay more standing capacity → recover faster and lose less. Memorize the four points on the spectrum AND the RPO/RTO bands — questions give you numbers and ask you to pick the cheapest strategy that meets them.

Strategy	What's running in DR Region	RPO / RTO	Cost
Backup & Restore	Nothing — backups replicated to S3 (cross-Region); rebuild with IaC when disaster hits	Hours / up to 24h+	$
Pilot Light	Data layer ONLY, always replicating (cross-Region read replica, Global Table); app servers exist as AMIs/templates, OFF	Minutes / tens of minutes–hours	$$
Warm Standby	A complete but SCALED-DOWN copy serving no (or test) traffic; scale up + flip DNS on failover	Seconds–minutes / minutes	$$$

Strategy	What's running in DR Region	RPO / RTO	Cost
Multi-site Active/Active	Full capacity in both Regions serving live traffic (Route 53 latency/weighted); failover = stop routing to the dead one	~0 / ~0 (seconds)	$$$$

- **AWS Elastic Disaster Recovery (DRS):** continuous block-level replication of physical/virtual/cloud servers into a cheap staging area; launch full recovery instances in minutes. The lift-and-shift DR answer with low RPO/RTO at low standing cost.
- Supporting cast: CloudFormation/IaC makes Backup & Restore and Pilot Light feasible (rebuild on demand); Route 53 failover routing performs the traffic flip; cross-Region snapshot copy automation closes the RPO gap.

EXAM TIP: Extract the numbers, then pick the CHEAPEST band that satisfies them. 'RPO 15 min, RTO 4 h, minimize cost' → Pilot Light. 'Must serve users during a Regional outage with no interruption' → Active/Active. 'Cheapest possible DR, next-day recovery OK' → Backup & Restore.

9. Reference Architecture: Resilient Order-Processing System

Flow: Users → Route 53 → ALB (3 AZs) → web ASG (private subnets, target tracking on requests/target) → SNS topic 'OrderPlaced' → three SQS queues (payments FIFO, fulfillment, analytics) → worker ASGs scaling on backlog-per-instance → Aurora Multi-AZ (writer + 2 readers) + DynamoDB session/cart table. DLQs on every queue; Step Functions orchestrates payment retries; AWS Backup handles snapshots; Aurora Global Database replicates to a second Region where a pilot-light copy waits behind a Route 53 failover record.

- Why each piece: ALB+ASG across 3 AZs = fault-tolerant web tier. SNS→SQS fan-out = one event, three independent consumers; a fulfillment outage can't lose orders (queue absorbs). FIFO for payments = no duplicate charges, ordered per customer. Backlog-based scaling = workers sized by actual work. Aurora readers = reports don't touch the writer. Failover record + Global DB = Region-level DR with ~1 s RPO.

Exam questions describe a broken version of this: synchronous web→worker calls that drop orders at peak (add SQS), one NAT/one AZ (add per-AZ), reports slowing production (read replica), no DLQ (poison messages loop forever).

10. Worked Practice Questions

Practice Q1. During flash sales, a web app writes orders directly to RDS. Spikes overwhelm the database and orders are lost. Users must receive immediate confirmation. What's the best fix?

✗ Bigger RDS instance — vertical scaling just raises the ceiling; spikes will find it again, and it costs 24/7.

✗ Aurora migration alone — faster DB, same tight coupling; extreme spikes still overwhelm synchronous writes.

✓ Put SQS between the web tier and a worker fleet: confirm the order once the message is queued, let workers write to the DB at a sustainable rate, scale workers on queue depth. Queue-as-shock-absorber is the signature Domain 2 answer.

Practice Q2. An application must process payment messages exactly once and in the order each customer submitted them. Which service?

✗ SQS Standard — at-least-once + best-effort ordering: duplicates and reordering possible.

✗ SNS — push fan-out, no ordering/consumption semantics for a single worker pipeline.

✓ SQS FIFO with MessageGroupId = customer ID — exactly-once processing, strict per-group ordering, and different customers still process in parallel.

Practice Q3. A company needs RPO of 1 second and RTO under 1 minute for its MySQL database across Regions. Which solution meets this?

✗ Cross-Region RDS read replica — async lag can be seconds–minutes; promotion is manual and slower than 1 minute in practice.

✗ Nightly snapshots copied cross-Region — RPO up to 24 hours.

✓ Aurora Global Database — storage-level replication ~1 s lag, secondary promotion <1 min. Those two numbers ARE the Aurora Global Database spec; recognize them instantly.

Practice Q4. A reporting team's heavy analytical queries are degrading a production RDS PostgreSQL database. Least-disruptive fix?

✗ Multi-AZ — the standby is not readable; Multi-AZ addresses availability, not read load.

✗ Second RDS instance with manual dumps — stale data and operational toil.

✓ Create a read replica and point reporting at it — async replication offloads reads; production writes untouched. Multi-AZ = availability; replicas = read scaling. The exam retests this distinction endlessly.

Practice Q5. An ASG behind an ALB replaces instances only when the VM fails, but sometimes the application hangs while the instance stays 'healthy'. Fix?

✗ Faster instance types — unrelated to detection.

✗ CloudWatch CPU alarm to reboot — a hung app may show normal CPU.

✓ Enable ELB health checks on the ASG — the ALB's HTTP health check sees the app not responding; the ASG then terminates and replaces it. EC2 checks watch the VM; ELB checks watch the APP.

Practice Q6. A stateless app runs in one AZ on 6 instances. The architect must make it survive an AZ outage with NO reduction in capacity and no manual action. What's correct?

✗ 6 instances in one AZ with an ASG — self-healing, but an AZ outage takes all capacity at once.

✗ 3+3 across two AZs — survives, but at 50% capacity until scale-out catches up: 'no reduction' fails.

✓ Spread MORE than required capacity across AZs so any single-AZ loss still leaves ≥6 (e.g., 3×3 across three AZs = 9, lose one → 6; or 6+6 across two). Fault tolerance = pre-paid redundancy; the ALB reroutes instantly.

Practice Q7. When a new object lands in an S3 bucket, three systems must react independently: thumbnail generation (Lambda), a compliance scan (Lambda), and an audit log (SQS → on-prem puller). Some consumers may be added later. Best design?

✗ Three separate S3 event notifications wired directly — S3 eventing per prefix/suffix is limited and adding consumers means touching bucket config each time.

✗ Lambda that calls the other systems synchronously — tight coupling; one failure blocks the rest.

✓ S3 → SNS topic (or EventBridge) → subscribers (both Lambdas + the SQS queue). Fan-out decouples consumers; adding a fourth is just a new subscription. 'Multiple independent reactions to one event' → SNS/EventBridge fan-out.

Practice Q8. A company wants active-passive failover from its primary Region to a static maintenance page when the primary fails, automatically. Simplest solution?

✗ Manual DNS change in an outage — not automatic; error-prone at 3 AM.

✗ Global Accelerator — fails over between healthy endpoints, but a static S3 page isn't a GA endpoint; heavier than needed.

✓ Route 53 FAILOVER routing: primary record with a health check on the ALB; secondary record aliased to a CloudFront/S3 static site. Health check fails → DNS serves the fallback automatically.

Self-check before Module 3

- HA vs fault tolerance — which one allows brief downtime, and what does the other cost you?
- Can you explain visibility timeout, at-least-once delivery, and why consumers must be idempotent?
- Multi-AZ vs read replica: sync vs async, readable vs not, what each solves — without looking?
- Recite the four DR strategies with RPO/RTO bands, cheapest to priciest.
- Draw the order-processing architecture and justify every arrow.